Learning Successor Measures from Manipulation Play

Anonymous authors
Paper under double-blind review

Abstract

[Caleb: not an abstract, just an outline of the intro] Zero shot manipulation is a highly desirable capability that FB models provide. However, FB models fail miserably at manipulation because of severe imprecision. In this work we expose the key limitation which explains this difficulty: bottleneck state misapplication of capacity grasping example: overly permissive when approaching the block, poor retention after grasping it We offer a simple and elegant solution that targets the core of this problem: Redistributing representational capacity to stabilize performance. by extending normalized successor-distribution learning to FB-style zero-shot control, this resolves both the core representational challenges and the zero shot control We show that this improves the performance of goal-conditioned benchmarks We also show that unseen sparse, dense and held-out rewards are possible from the same successor representation

Behavior foundation models (BFMs) offer the tantalizing promise of creating generally capable agents: simply pretrain policies using offline data that offers broad coverage, and recover optimal policies for a wide range of tasks. [Aditya: Need some citations here] For example, such pretraining can enable a robot to perform complex manipulations, even those not exactly demonstrated in the dataset. Successor-measure (SM) methods provide a natural framework for this goal. Forward-backward representations (FB) (Touati & Ollivier, 2021; Touati et al., 2023; Tirinzoni et al., 2025) and proto-successor measures (PSM) (Agarwal et al., 2025), for example, learn representations of the discounted future-state occupancy induced by a state, action, and policy. Because the value of any state-based reward can be obtained by integrating that reward against the corresponding successor measure, these representations can, in principle, support zero-shot policy recovery for rewards that were never specified during pretraining. Yet this promise remains difficult to realize when control requires precise interactions. In manipulation, we find that FB can reach the relevant object while failing at the interaction that determines task success, such as establishing a reliable grasp in cube pick-and-place, causing performance to deteriorate sharply as tasks require more precise or compositional behavior Mohan et al. (2025).

We hypothesize that this loss of precision arises at control bottlenecks: states where a small number of precise actions determine access to a much broader region of future behavior. Unlike classical TD learning, which compresses an action's consequences into a scalar value for a given reward, SM methods must represent the full distribution of possible future states. Consider the state immediately before grasping a cube. For many nearby future states, successful and unsuccessful grasping actions induce similar successor predictions because the cube has not yet moved. The distinction that matters for control appears primarily after the bottleneck: a successful grasp makes a large family of downstream cube configurations reachable, whereas a failed grasp does not. This action-discriminative signal is therefore distributed across many individual future states. Under finite-capacity SM-approximation, the model must simultaneously represent the large set of weakly informative futures and this diffuse but control-critical downstream region, potentially allocating insufficient capacity to the latter. This mismatch explains why SM models can accurately capture coarse future occupancy while still failing at the precise interactions that determine task success. More generally, the same failure mode can arise whenever a narrow set of actions controls access to a substantially larger region of future state space.

In this work, we provide a simple, elegant solution to this problem: rather than learning each future-state occupancy in isolation, we let possible futures compete for a fixed amount of successor probability. Our method, Density-FB, recasts the successor measure as a normalized distribution over future states and extends it to FB-style zero-shot control. Normalization preserves the information in the successor measure while changing what the model must learn: instead of matching the absolute magnitude of individual occupancy values, it must determine how to allocate future probability across competing outcomes. Concretely, Density-FB utilizes the FB bilinear product as a log-density score and replaces squared successor-density regression with a cross-entropy Bellman objective over the normalized distribution, thereby re-balancing how SM approximation allocates capacity across possible futures. This changes the relative importance of approximation errors across future states, reducing the dominance of concentrated successor regions and better preserving collectively important distinctions across diffuse post-bottleneck futures. Density-FB thereby retains the reusable successor representation central to FB while making it better suited to precision-sensitive control.

[Aditya: TODO: flesh this out with results. We show that this improves the performance of goal-conditioned benchmarks We also show that unseen sparse, dense and held-out rewards are possible from the same successor representation]

1 Preliminaries

Offline reward-free setting. We consider a discounted Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, \gamma)$ with $\gamma \in (0, 1)$. The agent is given a fixed dataset $\mathcal{D} = \{(s_i, a_i, s'_i)\}_{i=1}^{N}$ collected without downstream reward labels. We denote by ρ the marginal distribution of next states s' in $\mathcal{D}$. Downstream tasks are specified by rewards $r : \mathcal{S} \to \mathbb{R}$.

Successor measures. For a policy π, define its discounted successor measure over future states as

$$M^\pi(s, a, X) := \mathbb{E}_\pi \left[\sum_{t \geq 0} \gamma^t \mathbf{1}[s_{t+1} \in X] \,\middle|\, s_0 = s,\ a_0 = a \right], \tag{1}$$

for measurable $X \subseteq \mathcal{S}$. It satisfies the Bellman recursion

$$M^\pi(s, a, \cdot) = P(\cdot \mid s, a) + \gamma \mathbb{E}_{\substack{s' \sim P(\cdot|s,a) \\ a' \sim \pi(\cdot|s')}} \left[M^\pi(s', a', \cdot) \right]. \tag{2}$$

For any downstream reward r, integrating r against the successor measure recovers its action value,

$$Q_r^\pi(s, a) = \int_{\mathcal{S}} r(s^+)\, M^\pi(s, a, ds^+) = \mathbb{E}_\pi \left[\sum_{t \geq 0} \gamma^t r(s_{t+1}) \,\middle|\, s_0 = s,\ a_0 = a \right]. \tag{3}$$

Importantly, every successor measure has fixed total mass

$$M^\pi(s, a, \mathcal{S}) = \sum_{t \geq 0} \gamma^t = \frac{1}{1 - \gamma}. \tag{4}$$

2 Density-FB

Density-FB models the normalized successor measure as a conditional distribution over future outcomes and fits it with a categorical Bellman update. We first fix what is predicted (Section 2.1), then derive the update (Sections 2.2–2.3), relate it to TD InfoNCE (Section 2.4), and finally show how the learned distribution supplies a scalar critic for policy learning (Section 2.5) and a value for unseen rewards (Section 2.6). Throughout, the presentation follows the hard variant, which is the configuration we deploy; the smoothed variant we ablate against is stated in Section 2.7.

2.1 What is predicted

Write (s, a, s^+) for a transition in $\mathcal{D}$ and let $\phi : \mathcal{S} \to \mathcal{X}$ be a fixed outcome map, with

$$x := \phi(s), \qquad x^+ := \phi(s^+). \tag{5}$$

The two towers see different inputs: $F_\theta(s, a, z) \in \mathbb{R}^d$ receives the full state, the action, and a task index z, whereas $B_\theta(x) \in \mathbb{R}^d$ represents a candidate future outcome. The object we learn is therefore the pushforward $\phi_\# M^{\pi_z}$ of the successor measure of Equation (1), which we continue to write $M^{\pi_z}(s, a, \cdot)$ with the understanding that its argument is a set of outcomes. Pushforward preserves total mass, so Equations (2)–(4) carry over verbatim with $\mathcal{S}$ replaced by $\mathcal{X}$ and $P(\cdot \mid s, a)$ by its pushforward $P_\phi(\cdot \mid s, a)$. We write ρ_ϕ for the outcome marginal of ρ.

In our manipulation experiments ϕ extracts the object coordinate blocks, so the model conditions on the full state but predicts occupancy over the goal-relevant object configuration. This choice fixes the method's reward scope: Density-FB evaluates any bounded reward that is measurable on $\mathcal{X}$ and supported by ρ_ϕ — not arbitrary MDP rewards, which may depend on arm joint state, action cost, or path-dependent quantities that ϕ discards. It is also privileged information relative to a plain-FB baseline whose backward map receives the full observation: no reward or task labels are used, but pretraining is not domain-agnostic. Taking $\phi = \mathrm{id}$ recovers $x = s$ and asks B_θ to represent the full state.

2.2 Normalizing the successor measure

Since $M^{\pi_z}(s, a, \mathcal{X}) = 1/(1-\gamma)$ for every (s, a, z), the total mass carries no information. We divide it out and define the normalized successor measure

$$\nu^{\pi_z}(\cdot \mid s, a) := (1-\gamma)\, M^{\pi_z}(s, a, \cdot), \qquad \nu^{\pi_z}(\mathcal{X} \mid s, a) = 1. \tag{6}$$

Multiplying Equation (2) by $1-\gamma$ gives the recursion in normalized form,

$$\nu^{\pi_z}(\cdot \mid s, a) = (1-\gamma) P_\phi(\cdot \mid s, a) + \gamma\, \mathbb{E}_{\substack{s' \sim P(\cdot|s,a) \\ a' \sim \pi_z(\cdot|s')}} [\nu^{\pi_z}(\cdot \mid s', a')]\,. \tag{7}$$

The factor $1-\gamma$ appears only on the immediate term; the bootstrap coefficient is still γ. The immediate terms therefore cancel in a difference, and the induced operator is a γ-contraction in total variation (Appendix A.3), exactly as for the unnormalized measure.

Standard FB reads the bilinear product $F_\theta(s, a, z)^\top B_\theta(x)$ directly as a density of M^{π_z} with respect to ρ_ϕ, an unconstrained scalar. Density-FB instead treats the same quantity as a log-density score and normalizes it,

$$f_\theta(s, a, z, x) := \frac{F_\theta(s, a, z)^\top B_\theta(x)}{\tau}, \qquad h_\theta(s, a, z, x) := \frac{\exp f_\theta(s, a, z, x)}{\mathbb{E}_{\tilde{x} \sim \rho_\phi}[\exp f_\theta(s, a, z, \tilde{x})]}. \tag{8}$$

The model h_θ is a density ratio $d\nu^{\pi_z}/d\rho_\phi$ by construction: it is nonnegative and satisfies $\mathbb{E}_{\rho_\phi}[h_\theta] = 1$ for every (s, a, z), so the represented object is a probability distribution over outcomes rather than an unconstrained magnitude. This is the whole of the change to the representation; Appendix A.2 shows that the maximizer of the corresponding conditional objective is the true ratio.

Why normalization is the relevant constraint. Play data dwells: an object left in place produces many transitions whose successor outcome is the same. An unconstrained bilinear density can absorb repeated positive updates at such outcomes by increasing critic scale rather than by reallocating a fixed budget of future occupancy. Under Equation (8) that direction is closed — repeated stationary transitions can move where predicted occupancy sits, but they cannot increase its total. [Aditya: link back to the bottleneck framing in the intro once the intro is final.]

2.3 The hard categorical Bellman target

Equation (7) is an identity between measures; we turn it into a loss on sampled supports. For a minibatch $\{(s_i, a_i, s_i^+)\}_{i=1}^n$ with task indices z_i, build the query set from the batch's

own successors,

$$U = \{u_j\}_{j=1}^{n}, \qquad u_j := \phi(s_j^+), \tag{9}$$

which are n i.i.d. draws from ρ_ϕ. Scoring row i against every query gives the model's occupancy over U,

$$p_{ij} := \mathrm{softmax}_j(f_\theta(s_i, a_i, z_i, u_j)), \qquad p_{ij} \geq 0, \quad \textstyle\sum_j p_{ij} = 1. \tag{10}$$

The query set is redrawn every update, so n is a Monte-Carlo sample size, not a spatial resolution.

Two ingredients build the target. First, the observed successor u_i is a single sample from the immediate distribution $P_\phi(\cdot \mid s_i, a_i)$, and because U contains the batch successors it occupies slot i; the hard immediate label is therefore the one-hot vector

$$e_{ij} := \mathbf{1}[j = i]. \tag{11}$$

Second, the bootstrap term is evaluated by target networks $\bar{F}, \bar{B}$ at the next state and an on-policy action $a_i^+ = \mathrm{sg}[\pi_\omega(s_i^+, z_i)]$, over the same query set,

$$\bar{p}_{ij}^+ := \mathrm{softmax}_j\left(\frac{\bar{F}(s_i^+, a_i^+, z_i)^\top \bar{B}(u_j)}{\tau}\right). \tag{12}$$

Sharing the support between the two terms is what makes the mixture well defined. Equation (7) then gives the categorical target

$$p_i^{\mathrm{tgt}} := \mathrm{sg}\left[(1-\gamma)\, e_i + \gamma\, \bar{p}_i^+\right], \qquad \textstyle\sum_j p_{ij}^{\mathrm{tgt}} = (1-\gamma) + \gamma = 1, \tag{13}$$

and we fit the prediction to it with cross-entropy,

$$\mathcal{L}_{\mathrm{occ}} := -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{n} p_{ij}^{\mathrm{tgt}} \log p_{ij}. \tag{14}$$

Equations (10)–(14) define hard Density-FB. The observed successor receives $1-\gamma$ of the normalized occupancy and the remaining γ is inherited from the predicted occupancy after the next state. Appendix A.4 gives the population form of this update and shows that exact minimization implements one application of the γ-contractive operator.

What else changes relative to FB. Besides the represented object, two estimator choices accompany the change: the distributional target is fitted by cross-entropy rather than by FB's squared density regression, and the query support is built from batch successors so that the immediate one-hot label always lies in the support. We also drop FB's orthonormalization of B. The softmax does not orthogonalize B; rather, $F^\top B$ is no longer an estimate of a density, only the resulting probabilities are, so an orthonormality constraint has no target to enforce.

2.4 Relation to TD InfoNCE

The hard estimator is not new in isolation. Its target is the categorical Bellman update of TD InfoNCE (Zheng et al., 2024), which classifies the observed next state against negatives drawn from the state marginal and bootstraps with the stop-gradient softmax at the next state-action.

Proposition 1 (Reduction). With the one-hot immediate label of Equation (11), a query support shared between the immediate and bootstrap terms, and matched normalization conventions, $\mathcal{L}_{\mathrm{occ}}$ reduces to the categorical Bellman update underlying TD InfoNCE, evaluated on the outcome coordinates $x = \phi(s)$ and with the task variable carried in the forward tower.

We therefore do not claim the estimator as a contribution. The reduction holds under matched conventions rather than as an implementation identity: TD InfoNCE labels the dynamics next state in its immediate term but a marginal draw in its bootstrap term, uses

ℓ_2-normalized towers, and extracts its policy with a goal-contrastive cross-entropy plus goal-conditioned behavioral cloning over relabeled goals. It is likewise offline-capable, so the distinction is not offline versus online. The distinction is the interface: TD InfoNCE learns a raw-goal-conditioned policy and occupancy estimator, with no reward-prompted policy family and no deployed zero-shot mechanism for arbitrary rewards. Density-FB contributes the outcome-space construction of Section 2.1, the reward-integrating policy extraction of Section 2.5, and the task interface of Section 2.6.

2.5 Control from a successor distribution

A standard critic returns a scalar $Q(s, a, z)$ directly; Density-FB returns a distribution over candidate outcomes, so we first read a scalar out of it. For any bounded $r : \mathcal{X} \to \mathbb{R}$ define

$$q_r^{\pi_z}(s, a) := \mathbb{E}_{x\sim\rho_\phi}[h_z^\pi(s, a, x)\, r(x)] = (1 - \gamma)\, Q_r^{\pi_z}(s, a), \tag{15}$$

where $h_z^\pi = d\nu^{\pi_z}/d\rho_\phi$ (Appendix A.6). The constant $1 - \gamma$ is independent of the action, so exact density estimation preserves the action ranking of $Q_r^{\pi_z}$. Replacing h_z^π by the model and the expectation by the query pool gives the estimator every scalar in this paper is computed with,

$$\hat{q}_r(s, a, z) := \sum_j p_j(s, a, z)\, r(u_j), \qquad p_j(s, a, z) = \mathrm{softmax}_j(f_\theta(s, a, z, u_j))\,. \tag{16}$$

Rather than reading value from $F(s, a, z)^\top z$ as in standard FB, Density-FB evaluates an action by integrating the requested reward under its predicted successor distribution.

The readout is self-normalized importance sampling. Because $\nu(du \mid s, a) \propto e^{f_\theta(s,a,z,u)}\rho_\phi(du)$ and the queries are drawn from ρ_ϕ, Equation (16) is exactly self-normalized importance sampling of $\mathbb{E}_\nu[r]$ with proposal ρ_ϕ and unnormalized weights e^{f_θ} (Owen, 2013). Two consequences follow and hold for every readout below. The estimator is consistent as the pool grows but biased at any finite pool size, as all self-normalized estimators are, which is why we report pool-size curves rather than a single pool. And the standard quality diagnostic for such weights, their effective sample size (Kong, 1992), is a collapse check we monitor throughout training. The softmax weights thus play two roles at once: they are the learned successor probabilities and they are the importance weights of the readout.

Goal readout and actor. During pretraining each task index is a relabeled goal, $z = g$, drawn by hindsight from the play data; no reward or task labels are used. The reward attached to g is a tolerance kernel on outcomes, normalized over the same query pool,

$$k_g^\epsilon(u_j) := \mathrm{softmax}_j\left(-\tfrac{1}{2}\left\|(u_j - \phi(g))/\epsilon\right\|^2\right), \tag{17}$$

with per-block tolerances ϵ for multi-object goals, giving the scalar critic $q_g(s, a) := \hat{q}_{k_g^\epsilon}(s, a, g) = \sum_j p_j(s, a, g)\, k_g^\epsilon(u_j)$. Because k_g^ϵ is normalized over U, its effective base measure is the kernel times the query-pool density; the composition of the pool therefore enters the semantics of the readout, not only its variance. We train a behavior-regularized actor with

$$\mathcal{L}_\pi := -\frac{\mathbb{E}_{s,g}\left[q_g(s, \pi_\omega(s, g))\right]}{\mathrm{sg}[\mathbb{E}_{s,g}\left|q_g - \mathbb{E}[q_g]\right| + \varepsilon]} + \lambda_{\mathrm{BC}}\, \mathcal{L}_{\mathrm{BC}}(\pi_\omega). \tag{18}$$

The predicted occupancy has a large action-independent component while the actor needs only differences between actions, so q_g is centered before its magnitude normalizer is formed; the normalizer fixes the scale of the value term relative to behavior cloning. Gradients propagate through the action input of F_θ but never update the towers. The flagship extractor is a flow policy, for which $\mathcal{L}_{\mathrm{BC}}$ is a flow-matching plus distillation pair; Appendix C states it exactly, together with the two-head ensembling of F_θ used by every readout.

2.6 Zero-shot reward specification

The FB reward embedding does not transfer. Standard FB obtains a task vector from the linear identity

$$\int_{\mathcal{X}} r(x)\, F(s, a, z)^\top B(x)\, d\rho_\phi(x) = F(s, a, z)^\top\, \mathbb{E}_{\rho_\phi}[r\, B]\,, \tag{19}$$

which motivates $z_r \propto \mathbb{E}_{\rho_\phi}[rB]$. Density-FB does not preserve this identity, because $F^\top B$ now sits inside a normalized exponential. The failure is not merely one of convenience: the normalized model has an exact gauge freedom that the FB reward moment does not share. For any constant $b \in \mathbb{R}^d$,

$$\mathrm{softmax}_x\left(\frac{F^\top(B(x)+b)}{\tau}\right) = \mathrm{softmax}_x\left(\frac{F^\top B(x)}{\tau}\right), \qquad \mathbb{E}_{\rho_\phi}[r(B+b)] = \mathbb{E}_{\rho_\phi}[rB] + b\,\mathbb{E}_{\rho_\phi}[r]. \tag{20}$$

Two backward representations that define the same Density-FB successor distribution therefore induce different $\mathbb{E}_{\rho_\phi}[rB]$, so the FB reward moment is not an identifiable task index for the normalized model. Centering r removes this particular shift but does not restore Equation (19); the derivation is in Appendix B.1.

Reward value from the successor distribution. What does transfer is the readout itself, which needs no task vector. Given reward samples $r(u_1), \ldots, r(u_P)$ on a pool $U_P \sim \rho_\phi$ drawn once from the dataset, we estimate the unnormalized value at (s, a, z) by

$$\hat{Q}_r(s,a,z) := \frac{\hat{q}_r(s,a,z) \;+\; \big(r(x_0) - \kappa^\top r\big)}{1-\gamma}, \qquad x_0 = \phi(s), \quad \kappa_j = k^\epsilon_s(u_j), \tag{21}$$

where $\hat{q}_r$ is Equation (16) on the pool and κ is the same tolerance kernel of Equation (17) evaluated at the current outcome x_0 against the pool. The bracketed pair is a control variate: $\kappa^\top r$ is the pool's own estimate of the quantity $r(x_0)$ that we know exactly, so the pair has mean approximately zero and cancels the leading finite-pool error of $\hat{q}_r$; equivalently, it is the exact identity $\mathbb{E}[\sum_t \gamma^t (r(x_{t+1}) - r(x_t))] = (1-\gamma)Q_r - r(x_0)$ read backwards. The two halves are only meaningful together, and $\hat{Q}_r$ is linear in the reward samples for a fixed pool.

Selecting a policy. Equation (21) scores a reward but does not by itself produce a policy, since the actor is conditioned on z. We keep z in the space used during pretraining and select within the learned family: for candidate indices $\mathcal{Z}_{\mathrm{cand}}$ covered during pretraining and a policy-selection state distribution μ_{sel},

$$J_r(z) := \mathbb{E}_{s\sim\mu_{\mathrm{sel}}}\Big[\hat{Q}_r(s, \pi_\omega(s,z), z)\Big], \qquad \hat{z}_r \in \arg\max_{z\in\mathcal{Z}_{\mathrm{cand}}} J_r(z). \tag{22}$$

All learned parameters stay frozen and we execute $\pi_\omega(\cdot, \hat{z}_r)$. This selects the highest-valued policy in the learned family; unlike standard FB it does not assume that an arbitrary reward maps linearly to an optimal policy index. [Aditya: This is the load-bearing scope decision for the paper — see the note in Section 2.7.]

2.7 Variant: smoothed immediate targets

Hard Density-FB places the immediate $1-\gamma$ mass exactly on the observed successor. The one alternative we ablate replaces that one-hot by the tolerance kernel of Equation (17) evaluated at x_i^+,

$$p^{\mathrm{tgt}}_{i,\mathrm{soft}} = \mathrm{sg}\Big[(1-\gamma)\, k^\epsilon_{x_i^+} + \gamma\, \bar{p}_i^+\Big], \tag{23}$$

changing nothing else — in particular not the actor, whose goal weighting is kernel-based in both variants. This is not label smoothing but a different estimand: writing $K_{\rho,\epsilon}$ for the data-density-weighted kernel, the soft fixed point satisfies $\int r\, d\nu^\pi_K = \int (K_{\rho,\epsilon} r)\, d\nu^\pi$, so the soft model evaluates the smoothed reward $K_{\rho,\epsilon} r$ rather than r. Sums of rewards survive this smoothing, but sharp boundaries, conjunctions, maxima over goals, and avoidance terms do not. The hard variant is thus the semantically exact interface for the reward families we transfer to, which is why it is the configuration we deploy.

Algorithm. Algorithm 1 summarizes hard Density-FB training and zero-shot deployment.

Algorithm 1 Hard Density-FB

Require: Offline dataset $\mathcal{D}$, outcome map ϕ and tolerances ϵ, towers F_θ, B_θ with targets $\bar{F}, \bar{B}$, actor π_ω, discount γ, temperature τ, BC weight λ_{BC}, EMA rate η

for each training iteration do
  Sample transitions $\{(s_i, a_i, s_i^+)\}_{i=1}^n \sim \mathcal{D}$ and hindsight goals $\{g_i\}_{i=1}^n$
  Form the query set $u_j \leftarrow \phi(s_j^+)$ and scores p_{ij} ▷ Eq. (10)
  $a_i^+ \leftarrow \text{sg}[\pi_\omega(s_i^+, g_i)]$; compute $\bar{p}_{ij}^+$ ▷ Eq. (12)
  $p_i^{\text{tgt}} \leftarrow \text{sg}[(1-\gamma)e_i + \gamma \bar{p}_i^+]$ ▷ hard target, Eq. (13)
  Update F_θ, B_θ by minimizing $\mathcal{L}_{\text{occ}}$ ▷ Eq. (14)
  Compute $q_{g_i}(s_i, \pi_\omega(s_i, g_i))$ ▷ Eqs. (16), (17)
  Update π_ω by minimizing $\mathcal{L}_\pi$ ▷ Eq. (18)
  $\bar{F} \leftarrow (1-\eta)\bar{F} + \eta F_\theta$; $\bar{B} \leftarrow (1-\eta)\bar{B} + \eta B_\theta$
end for

Zero-shot deployment
Draw a pool $U_P \sim \rho_\phi$ from $\mathcal{D}$ and evaluate the reward samples $r(u_1), \ldots, r(u_P)$
Select $\hat{z}_r$ by Eq. (22) using the readout of Eq. (21)
Execute the frozen policy $\pi_\omega(\cdot, \hat{z}_r)$

References

S. Agarwal, H. Sikchi, P. Stone, and A. Zhang. Proto successor measure: Representing the behavior space of an rl agent. In Proceedings of the 42nd International Conference on Machine Learning (ICML'25), 2025.

Augustine Kong. A note on importance sampling using standardized weights. Technical Report 348, University of Chicago, Department of Statistics, 1992.

A. Mohan, C. Chuck, S. Agarwal, and A. Zhang. Is manipulation hard for successor measures? In RLBrew: Ingredients for Developing Generalist Agents Workshop (RLC 2025), 2025.

Art B. Owen. Monte Carlo Theory, Methods and Examples. 2013. Available at https://artowen.su.domains/mc/.

A. Tirinzoni, A. Touati, J. Farebrother, M. Guzek, A. Kanervisto, Y. Xu, A. Lazaric, and M. Pirotta. Zero-shot whole-body humanoid control via behavioral foundation models. In The Thirteenth International Conference on Learning Representations (ICLR'25). ICLR, 2025. Published online: iclr.cc.

A. Touati and Y. Ollivier. Learning one representation to optimize all rewards. In M. Ranzato, A. Beygelzimer, K. Nguyen, P. Liang, J. Vaughan, and Y. Dauphin (eds.), Proceedings of the 35th International Conference on Advances in Neural Information Processing Systems (NeurIPS'21). CurranAssociates, 2021.

A. Touati, J. Rapin, and Y. Ollivier. Does zero-shot reinforcement learning exist? In The Eleventh International Conference on Learning Representations (ICLR'23). ICLR, 2023. Published online: iclr.cc.

Chongyi Zheng, Ruslan Salakhutdinov, and Benjamin Eysenbach. Contrastive difference predictive coding. In International Conference on Learning Representations, 2024. arXiv:2310.20141.

A Theoretical Analysis of Density-FB

This appendix derives the Density-FB representation objective and characterizes its population fixed point. The analysis concerns policy evaluation for a fixed task-conditioned policy π_z. In particular, the contraction result below applies to the successor-density update with

π_z held fixed; it does not imply convergence of the coupled representation and actor updates used in practice.

Coordinates. The derivations below are written in state coordinates, with ρ the dataset next-state marginal. Every statement transfers verbatim to the outcome coordinates $x = \phi(s)$ of Section 2.1 by replacing $\mathcal{S}$ with $\mathcal{X}$, $P(\cdot \mid s, a)$ with its pushforward $P_\phi(\cdot \mid s, a)$, and ρ with ρ_ϕ: pushforward is a measurable map, so it preserves total mass, the Bellman recursion, and absolute continuity. We keep state coordinates here only to avoid carrying ϕ through every display.

Throughout, ρ denotes the dataset next-state marginal. We assume that, for the state-action pairs and policies of interest, $P(\cdot \mid s, a) \ll \rho$ and $\nu^{\pi_z}(s, a, \cdot) \ll \rho$. For the contraction argument, we additionally assume that the relevant density ratios are bounded. We also assume that the partition functions appearing below are finite and nonzero.

A.1 Normalized successor density

For a fixed policy π_z, recall the discounted successor measure

$$M^{\pi_z}(s, a, ds^+) := \mathbb{E}_{\pi_z}\left[\sum_{t=0}^{\infty} \gamma^t \delta_{s_{t+1}}(ds^+) \,\middle|\, s_0 = s,\ a_0 = a\right]. \tag{24}$$

Its total mass is $1/(1-\gamma)$. Density-FB works with the normalized successor measure

$$\nu^{\pi_z}(s, a, ds^+) := (1-\gamma) M^{\pi_z}(s, a, ds^+), \tag{25}$$

which is a probability measure over future states.

Multiplying the Bellman recursion for M^{π_z} by $1-\gamma$ gives

$$\nu^{\pi_z}(s, a, ds^+) = (1-\gamma) P(ds^+ \mid s, a) + \gamma \mathbb{E}_{\substack{s' \sim P(\cdot|s,a) \\ a' \sim \pi_z(\cdot|s')}} \left[\nu^{\pi_z}(s', a', ds^+)\right]. \tag{26}$$

Define the one-step transition density ratio

$$\chi(s, a, s^+) := \frac{dP(\cdot \mid s, a)}{d\rho}(s^+), \tag{27}$$

and the normalized successor-density ratio

$$h_z^\pi(s, a, s^+) := \frac{d\nu^{\pi_z}(s, a, \cdot)}{d\rho}(s^+). \tag{28}$$

Because ν^{π_z} is normalized,

$$\mathbb{E}_{s^+ \sim \rho}\left[h_z^\pi(s, a, s^+)\right] = 1. \tag{29}$$

Dividing Equation (26) by ρ yields the Bellman equation for the density ratio:

$$h_z^\pi(s, a, s^+) = (1-\gamma)\chi(s, a, s^+) + \gamma \mathbb{E}_{\substack{s' \sim P(\cdot|s,a) \\ a' \sim \pi_z(\cdot|s')}} \left[h_z^\pi(s', a', s^+)\right]. \tag{30}$$

Thus Density-FB can be viewed as policy evaluation in the space of normalized successor-density ratios.

A.2 Conditional density-ratio estimation

We next show why the normalized exponential model used by Density-FB recovers a density ratio.

Consider any probability measure $\mu \ll \rho$, and let

$$g(s^+) := \frac{d\mu}{d\rho}(s^+).$$

For a real-valued score f, define the normalized model

$$h_f(s^+) := \frac{\exp f(s^+)}{\mathbb{E}_{\tilde{s}^+ \sim \rho}\left[\exp f(\tilde{s}^+)\right]}. \tag{31}$$

Lemma 1 (Density-ratio identification). For fixed μ and ρ, consider

$$\mathcal{J}_{\text{ratio}}(f) := \mathbb{E}_{s^+\sim\mu}[f(s^+)] - \log \mathbb{E}_{s^+\sim\rho}\left[\exp f(s^+)\right]. \tag{32}$$

Then

$$\mathcal{J}_{\text{ratio}}(f) = C_\mu - \mathrm{KL}\left(\mu \,\|\, h_f \rho\right), \tag{33}$$

where $C_\mu = \mathbb{E}_\mu[\log g]$ does not depend on f. Hence any maximizer satisfies

$$h_f(s^+) = \frac{d\mu}{d\rho}(s^+) \tag{34}$$

almost everywhere under ρ.

Proof. Since

$$\log h_f(s^+) = f(s^+) - \log \mathbb{E}_\rho\left[e^f\right],$$

we have

$$\begin{aligned}\mathrm{KL}\left(\mu \,\|\, h_f\rho\right) &= \mathbb{E}_\mu\left[\log \frac{g(s^+)}{h_f(s^+)}\right] \\ &= \mathbb{E}_\mu[\log g(s^+)] - \mathbb{E}_\mu[f(s^+)] + \log \mathbb{E}_\rho[e^f].\end{aligned} \tag{35}$$

Rearranging gives Equation (33). Since KL divergence is minimized at zero, the optimum satisfies $h_f = g$. □

When a finite optimal score exists, Lemma 1 implies

$$f^*(s^+) = \log \frac{d\mu}{d\rho}(s^+) + c, \tag{36}$$

where the additive constant c disappears under normalization.

Density-FB applies this construction conditionally on (s, a, z). We use the bilinear score

$$f_\theta(s, a, z, s^+) := \frac{F_\theta(s, a, z)^\top B_\theta(s^+)}{\tau}, \tag{37}$$

which induces

$$h_\theta(s, a, z, s^+) := \frac{\exp f_\theta(s, a, z, s^+)}{\mathbb{E}_{\tilde{s}^+\sim\rho}\left[\exp f_\theta(s, a, z, \tilde{s}^+)\right]}. \tag{38}$$

Thus $F_\theta^\top B_\theta/\tau$ parameterizes a log density ratio, while its normalized exponential represents the successor-density ratio itself.

A.3 Density-FB as a Bellman operator

For fixed z and policy π_z, define the operator

$$(\mathcal{T}_z h)(s, a, s^+) := (1-\gamma)\chi(s, a, s^+) + \gamma \mathbb{E}_{\substack{s'\sim P(\cdot|s,a)\\ a'\sim\pi_z(\cdot|s')}}\left[h(s', a', s^+)\right]. \tag{39}$$

Equation (30) immediately shows that

$$h_z^\pi = \mathcal{T}_z h_z^\pi. \tag{40}$$

The operator also preserves normalization. If $\mathbb{E}_\rho[h(s, a, \cdot)] = 1$ for every (s, a), then

$$\begin{aligned}\mathbb{E}_{s^+\sim\rho}\left[(\mathcal{T}_z h)(s, a, s^+)\right] &= (1-\gamma)\mathbb{E}_\rho[\chi(s, a, s^+)] + \gamma \mathbb{E}_{s', a'}\mathbb{E}_\rho[h(s', a', s^+)] \\ &= (1-\gamma) + \gamma = 1.\end{aligned} \tag{41}$$

Proposition 1 (Contraction). On the space of bounded density-ratio functions equipped with the supremum norm, $\mathcal{T}_z$ is a γ-contraction:

$$\left\|\mathcal{T}_z h_1 - \mathcal{T}_z h_2\right\|_\infty \le \gamma \left\|h_1 - h_2\right\|_\infty. \tag{42}$$

Proof. The one-step term cancels, giving

$$\begin{aligned}\left|(\mathcal{T}_z h_1)(s,a,s^+) - (\mathcal{T}_z h_2)(s,a,s^+)\right| &= \gamma \left|\mathbb{E}_{s',a'}\left[h_1(s',a',s^+) - h_2(s',a',s^+)\right]\right| \\ &\le \gamma \mathbb{E}_{s',a'}\left[\left|h_1(s',a',s^+) - h_2(s',a',s^+)\right|\right] \\ &\le \gamma \left\|h_1 - h_2\right\|_\infty . \end{aligned} \tag{43}$$

Taking the supremum over (s,a,s^+) yields Equation (42). □

Corollary 1 (Unique fixed point). Under the assumptions above, $\mathcal{T}_z$ has a unique bounded fixed point, equal to the normalized successor-density ratio h_z^π. Consequently, exact repeated Bellman updates

$$h_{k+1} = \mathcal{T}_z h_k \tag{44}$$

converge to h_z^π for any bounded initialization.

This result characterizes the representation update as policy evaluation: for a fixed π_z, the desired solution is the normalized discounted future-state distribution induced by that policy.

A.4 Population Density-FB objective

We now connect the Bellman operator to the objective optimized by Density-FB. Let $\bar{h}$ be a fixed target density ratio satisfying $\mathbb{E}_\rho[\bar{h}(s,a,z,\cdot)] = 1$, and define

$$h_z^{\mathrm{tgt}}(s,a,s^+) := (1-\gamma)\chi(s,a,s^+) + \gamma \mathbb{E}_{\substack{s'\sim P(\cdot|s,a) \\ a'\sim\pi_z(\cdot|s')}}\left[\bar{h}(s',a',z,s^+)\right]. \tag{45}$$

By Equation (41), h_z^{tgt} is itself a normalized density ratio.

The corresponding population cross-entropy loss is

$$\mathcal{L}_{\mathrm{DFB}}^{\mathrm{pop}}(\theta;\bar{h}) := -\mathbb{E}_{\substack{z\sim p_Z \\ (s,a)\sim\mathcal{D}_{SA} \\ s^+\sim\rho}}\left[h_z^{\mathrm{tgt}}(s,a,s^+)\log h_\theta(s,a,z,s^+)\right]. \tag{46}$$

Proposition 2 (Bellman update by cross-entropy minimization). If the model class can represent h_z^{tgt} exactly, a global minimizer of Equation (46) satisfies

$$h_\theta(s,a,z,s^+) = h_z^{\mathrm{tgt}}(s,a,s^+) = (\mathcal{T}_z\bar{h})(s,a,s^+) \tag{47}$$

almost everywhere under $\mathcal{D}_{SA}\times\rho$.

Proof. For each fixed (s,a,z), Equation (46) is the cross-entropy between the target distribution $h_z^{\mathrm{tgt}}\rho$ and model distribution $h_\theta\rho$. It can be written as

$$H(h_z^{\mathrm{tgt}}\rho) + \mathrm{KL}\left(h_z^{\mathrm{tgt}}\rho \,\middle\|\, h_\theta\rho\right),$$

where the first term is independent of θ. The minimum is therefore attained when $h_\theta = h_z^{\mathrm{tgt}}$. □

Combining Proposition 1 and Proposition 2, exact optimization against the current density target implements one step of a γ-contractive policy-evaluation operator.

Remark (the practical update is a projected Bellman step). Proposition 2 assumes the model class contains h_z^{tgt}. The class actually used is the exponential-bilinear family

$$\mathcal{H}_d := \left\{ x \mapsto \exp\big(F(s,a,z)^\top B(x)/\tau\big)/Z(s,a,z) \;:\; F,B \text{ measurable},\; F,B\in\mathbb{R}^d \right\},$$

whose members have log-densities of rank at most d in (s,a,z) and x jointly. $\mathcal{H}_d$ is not closed under $\mathcal{T}_z$: the mixture $(1-\gamma)\chi + \gamma\bar{h}$ of a transition ratio and a rank-d log-linear model is generally not itself log-linear of rank d. The minimizer of Equation (46) over $\mathcal{H}_d$ is therefore the KL-projection of $\mathcal{T}_z\bar{h}$ onto $\mathcal{H}_d$ rather than $\mathcal{T}_z\bar{h}$ itself, and the practical iteration is $h_{k+1} = \Pi_{\mathcal{H}_d}\mathcal{T}_z h_k$. Since $\Pi_{\mathcal{H}_d}$ is a KL-projection and not a $\|\cdot\|_\infty$ non-expansion, Corollary 1 does not extend to this iteration; as with every projected Bellman method, convergence of

the practical update is not implied by the contraction of $\mathcal{T}_z$ alone. We state the exact-class results because they identify the intended fixed point, not because the parametric iteration inherits them.

To recover the score-space objective used in the derivation, substitute

$$\log h_\theta(s,a,z,s^+) = f_\theta(s,a,z,s^+) - \log Z_\theta(s,a,z),$$

where

$$Z_\theta(s,a,z) := \mathbb{E}_{s^+\sim\rho}\left[\exp f_\theta(s,a,z,s^+)\right]. \tag{48}$$

Since h_z^{tgt} integrates to one, maximizing the negative cross-entropy in Equation (46) is equivalent to

$$\begin{aligned}\mathcal{J}_{\mathrm{DFB}}(\theta;\bar{h}) = (1-\gamma)\mathbb{E}_{\substack{z\sim p_Z\\(s,a,s')\sim\mathcal{D}}}\left[f_\theta(s,a,z,s')\right]\\ + \gamma\mathbb{E}_{\substack{z\sim p_Z\\(s,a,s')\sim\mathcal{D}\\a'\sim\pi_z(\cdot|s')\\s^+\sim\rho}}\left[\bar{h}(s',a',z,s^+)f_\theta(s,a,z,s^+)\right]\\ - \mathbb{E}_{\substack{z\sim p_Z\\(s,a)\sim\mathcal{D}_{SA}}}\left[\log Z_\theta(s,a,z)\right].\end{aligned} \tag{49}$$

In practice, $\bar{h}$ is represented by target networks and treated as stop-gradient during this update.

A.5 Finite-pool approximation

The population normalization in Equation (38) requires expectations under ρ. Density-FB approximates these expectations using the next states in each minibatch.

Let

$$\{(s_i,a_i,s_i')\}_{i=1}^n \sim \mathcal{D}$$

be a minibatch. The candidate states $\{s_j'\}_{j=1}^n$ are samples from the dataset next-state marginal ρ. For task index z_i, define

$$f_{ij} := \frac{F_\theta(s_i,a_i,z_i)^\top B_\theta(s_j')}{\tau}, \qquad p_{ij} := \frac{\exp f_{ij}}{\sum_{k=1}^n \exp f_{ik}}. \tag{50}$$

Thus p_i is the model distribution over the empirical candidate pool.

Let

$$a_i' := \mathrm{sg}\left[\pi_\omega(s_i',z_i)\right] \tag{51}$$

and define the target-network distribution

$$\bar{p}_{ij}' := \mathrm{softmax}_j\left(\frac{\bar{F}(s_i',a_i',z_i)^\top \bar{B}(s_j')}{\tau}\right). \tag{52}$$

The observed transition successor s_i' provides the sample from the immediate transition distribution in Equation (26). The finite-pool Bellman target is therefore

$$p_{ij}^{\mathrm{tgt}} := \mathrm{sg}\left[(1-\gamma)\mathbf{1}[j=i] + \gamma\bar{p}_{ij}'\right]. \tag{53}$$

Both terms are normalized over j, so $\sum_j p_{ij}^{\mathrm{tgt}} = 1$.

The empirical representation loss is

$$\mathcal{L}_{\mathrm{occ}} := -\frac{1}{n}\sum_{i=1}^n\sum_{j=1}^n p_{ij}^{\mathrm{tgt}}\log p_{ij}. \tag{54}$$

Equation (54) is the finite-support counterpart of the population cross-entropy in Equation (46). As the candidate pool grows, the batch softmax provides a self-normalized Monte Carlo approximation to the conditional density model in Equation (38).

Two properties of this finite-pool construction are worth isolating.

The one-hot label. The hard immediate label $\mathbf{1}[j = i]$ is the empirical measure of a single draw $s'_i \sim P(\cdot \mid s_i, a_i)$ on the pool, so conditionally on (s_i, a_i) it is an unbiased estimate of the immediate term of Equation (26) evaluated on that support. Its placement in slot i is not a coincidence but a construction: the pool is built from the batch successors precisely so that the sampled immediate outcome is guaranteed to lie in the support shared with the bootstrap term. Had the pool been drawn independently of the transitions, the immediate outcome would generically be absent from it and the target would not be a distribution over the same n atoms.

Finite-pool bias. Both p_{ij} and $\bar{p}'_{ij}$ are self-normalized importance-sampling estimates: the numerator and the denominator are averages over the same n draws from ρ. Such estimators are consistent as $n \to \infty$ under the usual support and moment conditions, but they are biased at any finite n, with bias of order $1/n$ rather than zero. This is a property of the estimator class, not of the implementation, and it applies identically to the reward readouts of Appendix A.6, which are the same construction on a larger pool. The standard diagnostic for weight quality is the effective sample size $\mathrm{ESS}_i := 1/\sum_j p_{ij}^2$ (Kong, 1992), which we monitor during training as a collapse check on the learned weights.

A.6 Reward readout from the successor density

The learned density also provides a value estimate for arbitrary state rewards. For a fixed policy π_z and reward $r : \mathcal{S} \to \mathbb{R}$, define

$$q_r^{\pi_z}(s, a) := \mathbb{E}_{s^+ \sim \rho}\left[h_z^{\pi}(s, a, s^+) r(s^+)\right]. \tag{55}$$

Proposition 3 (Normalized reward value). The quantity in Equation (55) satisfies

$$q_r^{\pi_z}(s, a) = (1 - \gamma) Q_r^{\pi_z}(s, a), \tag{56}$$

where

$$Q_r^{\pi_z}(s, a) := \mathbb{E}_{\pi_z}\left[\sum_{t=0}^{\infty} \gamma^t r(s_{t+1}) \,\middle|\, s_0 = s,\ a_0 = a\right].$$

Proof. By the definition of the density ratio,

$$\begin{aligned} q_r^{\pi_z}(s, a) &= \int_{\mathcal{S}} r(s^+)\, d\nu^{\pi_z}(s, a, s^+) \\ &= (1 - \gamma) \int_{\mathcal{S}} r(s^+)\, dM^{\pi_z}(s, a, s^+) \\ &= (1 - \gamma)\mathbb{E}_{\pi_z}\left[\sum_{t=0}^{\infty} \gamma^t r(s_{t+1}) \,\middle|\, s_0 = s,\ a_0 = a\right]. \end{aligned} \tag{57}$$

□

Density-FB replaces the true density ratio with the learned model:

$$q_{\theta,r}(s, a, z) := \mathbb{E}_{s^+ \sim \rho}\left[h_\theta(s, a, z, s^+) r(s^+)\right]. \tag{58}$$

If $h_\theta = h_z^{\pi}$, then $q_{\theta,r} = q_r^{\pi_z} = (1 - \gamma) Q_r^{\pi_z}$. For an approximate successor model, $q_{\theta,r}$ is the corresponding model-based estimate.

Using a finite candidate pool $\{s_j^+\}_{j=1}^P$,

$$q_{\theta,r}(s, a, z) \approx \sum_{j=1}^{P} p_j(s, a, z) r(s_j^+), \tag{59}$$

where $p_j(s, a, z)$ is the normalized successor probability assigned by the model. Since $1 - \gamma$ is independent of the action, exact density estimation preserves the action ranking induced by $Q_r^{\pi_z}$.

A.7 The de-biased finite-pool readout

The readout deployed at evaluation time (Equation (21)) augments Equation (59) with a control variate. We record the identity it rests on.

Proposition 4 (Temporal-difference form of the reward value). For any bounded r and any policy π_z, with x_t the outcome sequence and x_0 the current outcome,

$$\mathbb{E}_{\pi_z}\left[\sum_{t\geq 0}\gamma^t\big(r(x_{t+1})-r(x_t)\big)\right] = (1-\gamma)\,Q_r^{\pi_z}(s,a) - r(x_0). \tag{60}$$

Proof. Split the second sum and reindex:

$$\mathbb{E}\left[\sum_{t\geq 0}\gamma^t r(x_t)\right] = r(x_0)+\mathbb{E}\left[\sum_{t\geq 1}\gamma^t r(x_t)\right] = r(x_0)+\gamma\,\mathbb{E}\left[\sum_{t\geq 0}\gamma^t r(x_{t+1})\right] = r(x_0)+\gamma\,Q_r^{\pi_z}(s,a).$$

Subtracting from $Q_r^{\pi_z} = \mathbb{E}[\sum_{t\geq 0}\gamma^t r(x_{t+1})]$ gives Equation (60). □

Rearranging Equation (60) and using $q_r^{\pi_z} = (1-\gamma)Q_r^{\pi_z}$ from Proposition 3,

$$Q_r^{\pi_z}(s,a) = \frac{r(x_0) + \big(q_r^{\pi_z}(s,a) - r(x_0)\big)}{1-\gamma}, \tag{61}$$

which is an identity, not yet an estimator. The estimator is obtained by replacing $q_r^{\pi_z}$ with the pool readout $\hat{q}_r = \sum_j p_j r(u_j)$ and — this is the operative choice — replacing the subtracted $r(x_0)$ by the pool's own estimate of it, $\kappa^\top r$ with $\kappa_j = k^{\epsilon}_{x_0}(u_j)$, while keeping the exact $r(x_0)$ outside:

$$\hat{Q}_r(s,a,z) = \frac{r(x_0) + \big(\hat{q}_r(s,a,z) - \kappa^\top r\big)}{1-\gamma}. \tag{62}$$

The added quantity $r(x_0) - \kappa^\top r$ is the error the same pool, kernel, and normalization machinery commit on a query whose exact answer is known. It has approximately zero mean and is positively correlated with the error of $\hat{q}_r$, so it acts as a control variate that removes the leading finite-pool term; it does not change the estimand. Two consequences follow. First, the two halves are meaningless separately: dropping $r(x_0)$ while keeping $-\kappa^\top r$ (or the reverse) introduces a first-order bias rather than removing one. Second, for a fixed pool both p and κ are independent of r, so Equation (62) is a dot product of a fixed vector against the reward samples and is exactly linear in r.

B Relation to Standard Forward-Backward Representations

B.1 Why the standard FB reward map does not transfer

Standard FB represents the density of the unnormalized successor measure using

$$\frac{dM^{\pi_z}(s,a,\cdot)}{d\rho}(s^+) \approx F(s,a,z)^\top B(s^+). \tag{63}$$

The linear factorization gives

$$\int_{\mathcal{S}} r(s^+)F(s,a,z)^\top B(s^+)\,d\rho(s^+) = F(s,a,z)^\top \mathbb{E}_{s^+\sim\rho}\left[r(s^+)B(s^+)\right]. \tag{64}$$

This identity motivates the standard FB reward embedding

$$z_r \propto \mathbb{E}_\rho[rB].$$

Density-FB changes the role of the bilinear product. Under Equation (38), $F^\top B/\tau$ is a log-score inside a normalized exponential, so the linear identity in Equation (64) no longer holds.

There is also an exact ambiguity in the backward representation. Consider any constant vector $b \in \mathbb{R}^d$ and the transformation

$$B(s^+) \mapsto B(s^+) + b. \tag{65}$$

For fixed (s, a, z), this changes every score by the same amount,

$$\frac{F(s,a,z)^\top (B(s^+) + b)}{\tau} = \frac{F(s,a,z)^\top B(s^+)}{\tau} + \frac{F(s,a,z)^\top b}{\tau}.$$

The additive term does not depend on s^+ and therefore cancels under normalization:

$$\frac{\exp(F^\top (B(s^+) + b)/\tau)}{\mathbb{E}_\rho[\exp(F^\top (B + b)/\tau)]} = \frac{\exp(F^\top B(s^+)/\tau)}{\mathbb{E}_\rho[\exp(F^\top B/\tau)]}. \tag{66}$$

The standard FB reward moment is not invariant to the same transformation:

$$\mathbb{E}_\rho\left[r(s^+)(B(s^+) + b)\right] = \mathbb{E}_\rho\left[r(s^+)B(s^+)\right] + b\,\mathbb{E}_\rho[r(s^+)]. \tag{67}$$

Hence two backward representations that define exactly the same Density-FB successor distribution can produce different values of $\mathbb{E}_\rho[rB]$. The standard FB reward moment is therefore not an identifiable task index for the normalized model.

Centering the reward removes the particular shift term in Equation (67), but it does not recover Equation (64): reward evaluation remains nonlinear in the backward features because $F^\top B$ appears inside a normalized exponential. Density-FB therefore requires a task-selection mechanism distinct from the standard FB linear reward embedding.

B.2 Reward-based selection within the learned policy family

Rather than constructing a new task vector in backward-feature coordinates, Density-FB keeps the task index in the same space used during pretraining. Let $\mathcal{Z}_{\text{cand}}$ be a finite set of task indices drawn from the policy family covered during training. For a downstream reward r, define

$$J_{\theta,r}(z) := \mathbb{E}_{s\sim\mu_{\text{sel}}}\left[q_{\theta,r}(s, \pi_\omega(s,z), z)\right]. \tag{68}$$

We select

$$\hat{z}_r \in \arg\max_{z\in\mathcal{Z}_{\text{cand}}} J_{\theta,r}(z) \tag{69}$$

and execute the frozen policy $\pi_\omega(\cdot, \hat{z}_r)$.

This procedure uses the downstream reward only to evaluate policies through their predicted successor distributions. It therefore does not require the reward itself to share coordinates with the backward representation. The resulting guarantee is correspondingly weaker than the idealized standard FB interface: Equation (69) identifies the highest-valued policy within the covered candidate family, not necessarily a globally optimal policy for an arbitrary reward.

C Implementation Details

This appendix states the parts of Density-FB that Section 2 abstracts: the outcome map actually used, the two-head ensembling of the forward tower, the exact policy-extraction losses, and the zero-shot evaluation protocol. Hyperparameters are collected in Table 1.

C.1 Outcome map and tolerances

The outcome map ϕ concatenates a fixed list of observation coordinate blocks, one per manipulable object, each with its own tolerance ϵ_b. A block is written start:stop:ϵ over observation indices. The backward tower receives the tolerance-scaled coordinates $\left(x_b/\epsilon_b\right)_b$, and the tolerance kernel of Equation (17) applies the same per-block scaling before taking the squared distance:

$$k_y^\epsilon(u_j) = \text{softmax}_j\left(-\tfrac{1}{2}\sum_b \left\|(u_{j,b} - y_b)/\epsilon_b\right\|^2\right). \tag{70}$$

The block lists are given in Table 1. The same ϵ_b serve three roles — backward-tower input scaling, the actor's goal weighting, and the control variate of Equation (21) — so a single tolerance per object is the only geometric quantity the method receives.

C.2 Networks and ensembling

Both towers are MLPs with the layer convention Linear $\to$ GELU $\to$ LayerNorm per hidden layer and a bare linear output; there is no output normalization on B_θ and no orthonormality penalty anywhere. F_θ takes the concatenation $[s, a, z]$ — in the goal-conditioned flagship, z is the raw goal observation — and is a two-head ensemble, $F_\theta(s, a, z) \in \mathbb{R}^{2\times d}$. B_θ is a single tower. Target towers $\bar{F}, \bar{B}$ are copies updated by an exponential moving average with rate η after every step; the actor has no target network, and the bootstrap action is taken from the live actor under a stop-gradient.

The two heads are combined only at readout time, and the combination is taken over probability vectors rather than over scalars: writing $p^{(1)}, p^{(2)}$ for the two heads' softmax weights over the pool,

$$p := \min\left(p^{(1)}, p^{(2)}\right) \quad \text{elementwise.} \tag{71}$$

This is the conservative choice used by both the actor and the zero-shot evaluation. It is worth stating plainly that the elementwise minimum of two simplex vectors is sub-normalized whenever the heads disagree, $\sum_j p_j \le 1$, so what the readouts compute is a sub-normalized version of the self-normalized estimator of Equation (16); the consistency statement of Section 2.5 is for single-head weights. We report this rather than silently renormalize, because every deployed number uses this path. The representation loss $\mathcal{L}_{\text{occ}}$ is averaged over the two heads, each fitted against its own target-network bootstrap.

C.3 Policy extraction

Let $q := \min(q^{(1)}, q^{(2)})$ be the two-head goal readout of Section 2.5 and

$$s_q := \text{sg}[\mathbb{E}\,|q - \mathbb{E}[q]| + \varepsilon]\,, \qquad \varepsilon = 10^{-6}, \tag{72}$$

its centered magnitude normalizer. Centering matters because the predicted occupancy carries a large action-independent component that would otherwise dominate the scale.

Flow actor (flagship). A one-shot actor $\pi_\omega(s, g; \eta)$ maps noise $\eta \sim \mathcal{N}(0, I)$ to an action, and a separate goal-agnostic velocity field $v_\psi(s, a, t)$ is trained by flow matching on the dataset actions. With $x_0 \sim \mathcal{N}(0, I)$, $x_1 = a$, $t \sim \mathcal{U}[0, 1]$ and $x_t = (1 - t)x_0 + tx_1$,

$$\mathcal{L}_{\text{flow}} := \mathbb{E}\big\|v_\psi(s, x_t, t) - (x_1 - x_0)\big\|^2. \tag{73}$$

Let $\text{ODE}_\psi(s, \eta)$ denote the K-step Euler rollout of v_ψ from η, taken without gradient. The full actor objective is

$$\mathcal{L}_\pi = -\frac{\mathbb{E}\left[q\big(s, \pi_\omega(s, g; \eta)\big)\right]}{s_q} \;+\; \lambda_{\text{BC}}\,\mathbb{E}\big\|\pi_\omega(s, g; \eta) - \text{ODE}_\psi(s, \eta)\big\|^2 \;+\; \mathcal{L}_{\text{flow}}, \tag{74}$$

i.e. the distillation term is scaled against the normalized value term while the flow-matching term, which trains a different network, is not. This is the FQL-style extractor; the behavior-cloning baseline is Equation (74) with the value term removed.

Gaussian DDPG+BC actor (alternative extractor). With $\pi_\omega(\cdot \mid s, g) = \mathcal{N}(\mu_\omega(s, g), \sigma^2 I)$ at fixed σ,

$$\mathcal{L}_\pi = -\frac{\mathbb{E}\left[q(s, \mu_\omega(s, g))\right]}{s_q} - \alpha\,\mathbb{E}\left[\log \pi_\omega(a \mid s, g)\right]. \tag{75}$$

Gradient hygiene. The value term differentiates through the action input of F_θ, so the towers appear in the actor graph. Their optimizers are never stepped on the actor loss and their gradients are cleared before the next representation update, so no actor gradient reaches F_θ or B_θ. The kernel weights k_g^ϵ are stop-gradded: the tolerance kernel is fixed geometry, not a trained object.

Table 1: Density-FB hyperparameters. Values are shared across environments unless stated.

	Symbol	Value
Successor model		
Latent dimension	d	50
Score temperature	τ	1.0
Tower hidden sizes	–	$(512, 512, 512)$, GELU + LayerNorm
Forward-tower heads	–	2 (elementwise min at readout, Eq. (71))
Backward-tower output norm	–	none
Orthonormality penalty	–	none
Immediate target	–	hard (one-hot); soft kernel is the ablation
Optimization		
Discount	γ	0.99
Batch size = training query-set size	n	256
Gradient steps	–	10^6
Optimizer	–	Adam, no weight decay
Learning rate (F, B, actor, velocity field)	–	3×10^{-4}
Target EMA rate	η	0.005
Policy extraction		
Extractor	–	one-shot flow actor (Gaussian DDPG+BC as alternative)
Actor hidden / layers	–	512, 2 hidden + 2 embedding layers
Velocity-field hidden / layers	–	512, 4
Euler steps in ODE_ψ	K	10
Distillation weight	λ_{BC}	3.0 (manipulation); 0.3 (antmaze)
DDPG+BC weight	α	0.3
Policy std (DDPG+BC)	σ	0.2, constant
Value centering	–	on
Zero-shot evaluation		
Default pool size	P	4096
Pool sweep	P	256, 1024, 4096, 16384
Episodes per task	–	25
Outcome map ϕ (blocks start:stop:ϵ over observation indices)		
cube-single	–	19:22:0.4
cube-double	–	19:22:0.4, 28:31:0.4
scene	–	19:22:0.4, 28:30:0.5, 32:34:0.5, 36:37:0.72, 38:39:0.6

C.4 Zero-shot evaluation protocol

A pool $U_P = \{u_1, \ldots, u_P\}$ is drawn once from the dataset outcome marginal ρ_ϕ with a fixed seed and its backward embeddings $B_\theta(u_j)$ are precomputed. Because P is a Monte-Carlo sample size and not a spatial resolution, evaluation may use a far larger pool than training ever saw; growing P reduces the self-normalization error of Equation (16) without changing the represented space. We report a pool sweep in addition to the default P.

The reward enters only as the sample vector $\big(r(u_1), \ldots, r(u_P)\big)$ together with $r(x_0)$; no reward labels, gradients, or fine-tuning are used. For fixed (s, a, z) and pool, both p and κ are independent of r, so the readout of Equation (21) is exactly a dot product of an r-independent vector against the reward samples — linear in r for any r, in exact arithmetic. The identity underlying the control variate is derived in Appendix A.7.

C.5 Hyperparameters